

```

let state = {
  baseline: 0,
  p1: { name: "Person 1", income: 0, freq: "biweekly", nextpay: "", tax: 0 },
  p2: { name: "Person 2", income: 0, freq: "biweekly", nextpay: "", tax: 0 },
  bills: []
};
/* TAB SWITCHING */
function showTab(id) {
  document.querySelectorAll(".tab").forEach(t => t.classList.remove("active"));
  document.querySelectorAll("[id^='tab-']").forEach(div => div.style.display = "none");
  document.querySelector(`button[onclick="showTab('${id}')"`).classList.add("active");
  document.getElementById(`tab-${id}`).style.display = "block";
  if (id === "result") renderResult();
}
/* FREQUENCY + NET CALC */
function checksPerMonth(freq) {
  return {
    biweekly: 2.167,
    weekly: 4.333,
    semimonthly: 2,
    monthly: 1
  }[freq] || 2;
}
function monthlyNet(person) {
  const gross = person.income * checksPerMonth(person.freq);
  return gross * (1 - person.tax / 100);
}
/* BILL ASSIGNMENT RULES */
function assignedToP1(b) {
  return b.account === "USAA" || b.account === "Cash";
}
function assignedToP2(b) {
  return b.account === "Chase" || b.account === "CapOne";
}
/* BILL RANGE */
function billsInRange(start, end) {
  return state.bills
    .filter(b => b.due >= start && b.due <= end)
    .reduce((s, b) => s + b.amt, 0);
}
/* SAVE */
function save() {
  state.baseline = parseFloat(document.getElementById("baseline-val").value) || 0;
  state.p1 = {
    name: document.getElementById("p1name").value || "Person 1",
    income: parseFloat(document.getElementById("p1income").value) || 0,
    freq: document.getElementById("p1freq").value,
    nextpay: document.getElementById("p1nextpay").value,
    tax: parseFloat(document.getElementById("p1tax").value) || 0
  };
  state.p2 = {
    name: document.getElementById("p2name").value || "Person 2",
    income: parseFloat(document.getElementById("p2income").value) || 0,
    freq: document.getElementById("p2freq").value,
    nextpay: document.getElementById("p2nextpay").value,
    tax: parseFloat(document.getElementById("p2tax").value) || 0
  };
  localStorage.setItem("hb-state5", JSON.stringify(state));
  renderIncomeSummary();
}

```

```

    renderBills();
    renderFlow();
    renderResult();
}
/* LOAD */
function load() {
    const saved = localStorage.getItem("hb-state5");
    if (saved) state = JSON.parse(saved);
    applyState();
    renderIncomeSummary();
    renderBills();
    renderFlow();
    renderResult();
}
/* APPLY STATE */
function applyState() {
    document.getElementById("baseline-val").value = state.baseline;
    document.getElementById("plname").value = state.pl.name;
    document.getElementById("plincome").value = state.pl.income;
    document.getElementById("plfreq").value = state.pl.freq;
    document.getElementById("plnextpay").value = state.pl.nextpay;
    document.getElementById("pltax").value = state.pl.tax;
    document.getElementById("p2name").value = state.p2.name;
    document.getElementById("p2income").value = state.p2.income;
    document.getElementById("p2freq").value = state.p2.freq;
    document.getElementById("p2nextpay").value = state.p2.nextpay;
    document.getElementById("p2tax").value = state.p2.tax;
}
/* ADD BILL */
function addBill() {
    const name = document.getElementById("b-name").value.trim();
    const amt = parseFloat(document.getElementById("b-amt").value) || 0;
    const due = parseInt(document.getElementById("b-due").value) || 1;
    const account = document.getElementById("b-account").value;
    if (!name || !amt) return;
    state.bills.push({ id: Date.now(), name, amt, due, account });
    document.getElementById("b-name").value = "";
    document.getElementById("b-amt").value = "";
    document.getElementById("b-due").value = "";
    save();
}
/* RENDER BILLS */
function renderBills() {
    const tbody = document.getElementById("bills-body");
    if (!state.bills.length) {
        tbody.innerHTML = "<tr><td colspan='5'>No bills yet.</td></tr>";
        return;
    }
    tbody.innerHTML = state.bills
        .sort((a,b)=>a.due-b.due)
        .map(b => `
        <tr>
            <td>${b.name}</td>
            <td>${b.amt.toFixed(2)}</td>
            <td>${b.due}</td>
            <td>${b.account}</td>
            <td><button class="btn btn-danger" onclick="deleteBill(${b.id})">X</button></td>
        </tr>
    `).join("");
}

```

```

}
function deleteBill(id) {
  state.bills = state.bills.filter(b => b.id !== id);
  save();
}
/* INCOME SUMMARY */
function renderIncomeSummary() {
  const m1 = monthlyNet(state.p1);
  const m2 = monthlyNet(state.p2);
  document.getElementById("income-summary").innerHTML = `
    <div class="metric"><div class="metric-label">${state.p1.name} monthly net</div><div class="
metric-value">${m1.toFixed(2)}</div></div>
    <div class="metric"><div class="metric-label">${state.p2.name} monthly net</div><div class="
metric-value">${m2.toFixed(2)}</div></div>
    <div class="metric"><div class="metric-label">Combined monthly net</div><div class="metric-
value">${(m1+m2).toFixed(2)}</div></div>
    <div class="metric"><div class="metric-label">Starting balance</div><div class="metric-valu
e">${state.baseline.toFixed(2)}</div></div>
  `;
}
/* CASH FLOW */
function getPayDatesThisMonth(p) {
  const now = new Date();
  const year = now.getFullYear();
  const month = now.getMonth();
  const dates = [];
  if (!p.nextpay) return dates;
  const step = {
    biweekly: 14,
    weekly: 7,
    semimonthly: 15,
    monthly: 30
  }[p.freq] || 14;
  let d = new Date(p.nextpay + "T00:00:00");
  while (d <= new Date(year, month + 1, 0)) {
    if (d.getMonth() === month) dates.push(new Date(d));
    d = new Date(d.getTime() + step * 86400000);
  }
  return dates.sort((a, b) => a - b);
}
function renderFlow() {
  const now = new Date();
  const year = now.getFullYear();
  const month = now.getMonth();
  const events = [];
  getPayDatesThisMonth(state.p1).forEach(d => {
    events.push({
      day: d.getDate(),
      label: `${state.p1.name} paycheck`,
      amount: state.p1.income,
      account: "USAA"
    });
  });
  getPayDatesThisMonth(state.p2).forEach(d => {
    events.push({
      day: d.getDate(),
      label: `${state.p2.name} paycheck`,
      amount: state.p2.income,
      account: "USAA"
    });
  });
}

```

```

    });
  });
  state.bills.forEach(b => {
    events.push({
      day: b.due,
      label: b.name,
      amount: -b.amt,
      account: b.account
    });
  });
  events.sort((a, b) => a.day - b.day);
  let usaa = state.baseline;
  let chase = 0;
  let rows = "";
  events.forEach(e => {
    if (e.account === "USAA") usaa += e.amount;
    if (e.account === "Chase") chase += e.amount;
    rows += `
      <tr>
        <td>${e.day}</td>
        <td>${year}-${String(month+1).padStart(2,"0")}-${String(e.day).padStart(2,"0")}</td>
        <td>${e.label}</td>
        <td>${e.amount > 0 ? "+" : ""}${Math.abs(e.amount).toFixed(2)}</td>
        <td>${usaa.toFixed(2)}</td>
        <td>${chase.toFixed(2)}</td>
      </tr>
    `;
  });
  document.getElementById("flow-body").innerHTML = rows;
  document.getElementById("flow-summary").innerHTML = `
    <div class="metric"><div class="metric-label">Starting balance</div><div class="metric-value">
    e">${state.baseline.toFixed(2)}</div></div>
    <div class="metric"><div class="metric-label">End of month USAA</div><div class="metric-value">
    ue">${usaa.toFixed(2)}</div></div>
  `;
}
/* RESULT TAB */
function renderResult() {
  const m1 = monthlyNet(state.p1);
  const m2 = monthlyNet(state.p2);
  const assigned1 = state.bills.filter(assignedToP1).reduce((s,b)=>s+b.amt,0);
  const assigned2 = state.bills.filter(assignedToP2).reduce((s,b)=>s+b.amt,0);
  const remain1 = m1 - assigned1;
  const remain2 = m2 - assigned2;
  const first15 = billsInRange(1, 15);
  const lastHalf = billsInRange(16, 31);
  document.getElementById("result-summary").innerHTML = `
    <div class="metric">
      <div class="metric-label">${state.p1.name} monthly net</div>
      <div class="metric-value">${m1.toFixed(2)}</div>
    </div>
    <div class="metric">
      <div class="metric-label">${state.p1.name} assigned bills</div>
      <div class="metric-value">${assigned1.toFixed(2)}</div>
    </div>
    <div class="metric">
      <div class="metric-label">${state.p1.name} remaining</div>
      <div class="metric-value">${remain1.toFixed(2)}</div>
    </div>
  `;
}

```

```

    <div class="metric">
      <div class="metric-label">${state.p2.name} monthly net</div>
      <div class="metric-value">${m2.toFixed(2)}</div>
    </div>
    <div class="metric">
      <div class="metric-label">${state.p2.name} assigned bills</div>
      <div class="metric-value">${assigned2.toFixed(2)}</div>
    </div>
    <div class="metric">
      <div class="metric-label">${state.p2.name} remaining</div>
      <div class="metric-value">${remain2.toFixed(2)}</div>
    </div>
    <div class="metric">
      <div class="metric-label">Bills 1-15</div>
      <div class="metric-value">${first15.toFixed(2)}</div>
    </div>
    <div class="metric">
      <div class="metric-label">Bills 16-EOM</div>
      <div class="metric-value">${lastHalf.toFixed(2)}</div>
    </div>
  `;
}
/* EXPORT */
function exportData() {
  const dataStr = JSON.stringify(state, null, 2);
  const blob = new Blob([dataStr], { type: "application/json" });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = "budget-data.json";
  a.click();
  URL.revokeObjectURL(url);
}
/* IMPORT */
function importData(file) {
  const reader = new FileReader();
  reader.onload = () => {
    try {
      state = JSON.parse(reader.result);
      localStorage.setItem("hb-state5", JSON.stringify(state));
      applyState();
      renderIncomeSummary();
      renderBills();
      renderFlow();
      renderResult();
    } catch (e) {
      console.log("Invalid file");
    }
  };
  reader.readAsText(file);
}
/* INIT */
load();

```